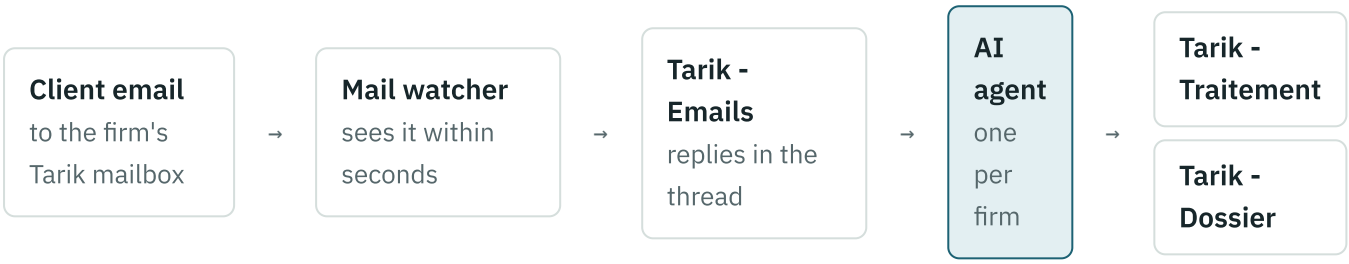
TARIK · HOW IT WORKS · FOR THE PROJECT TEAM

Tarik, the bookkeeping assistant for accounting firms

Clients email Tarik their invoices, bank statements and accounting exports. Tarik records them in Experio, the firm's accounting software, and replies in the same email thread. An AI agent understands each request; three fixed workflows in n8n do the work.

How the pieces fit together



The AI never writes to Experio itself. It reads the request, picks a workflow and fills in a few fields (which client file, purchase or sale). Every write goes through fixed, tested steps, and each step's result is recorded, so nothing is invented and everything can be traced.

FRONT DOOR

Tarik - Emails

Answers the firm's mailbox.

1. The mail watcher files each new email into its thread and saves the attachments.
2. Only the firm's own staff and Experio's own team, registered in Experio, are answered, and only at approved addresses. Anyone else is ignored and the manager is told.
3. A new conversation gets an instant reply: the message is received and Tarik answers in a few minutes.
4. The firm's AI agent reads the email and calls the right workflow.
5. The answer goes back as a reply in the same thread.

> **Follow-up email.** When Experio is still working (reading a scan, taking in imported entries), Tarik replies with what is done, then sends the rest in the same thread, within about 15 minutes. The client does not have to ask again.

INVOICES AND BANK STATEMENTS

Tarik - Traitement

Turns the attached documents into posted entries in Experio.

1. Identify each file: invoice or bank statement.
2. A bank statement is imported into the client file's bank account.
3. An invoice PDF is split if it holds several, and each becomes a draft in Experio.
4. Wait for Experio's own reading (OCR) of the draft.
5. Check and correct what was read; purchase or sale; supplier; duplicates; account (e.g. `6125`); VAT; journal; withholding tax; prepaid; fiscal year.
6. Post the invoice.

> **When it needs the client,** it asks one clear question marked “QUESTION :”. The client answers by email and processing continues from that exact step.

NEW CLIENT FILE

Tarik - Dossier

Opens a client file in Experio, or loads past accounting into one.

1. Create the file. Experio checks the name is free and the `ICE` and `IF` against the official register; a very similar name is confirmed with the client first.
2. Copy another file's setup (chart of accounts, settings), only if the client asks.
3. Import the history the client attached: a `FEC` or a general ledger. Experio creates the missing accounts, journals, suppliers, customers and fiscal years.
4. Finish the onboarding. Experio only takes lines once they are a minute old, so this step waits.
5. Report the file's state: accounts, journals, entries, open fiscal years.

> **Large imports** that cannot finish in time are completed by the follow-up email, which also sends the file's state.

CHANGING A CLIENT FILE

Tarik - Dossier, update mode

The same workflow updates an existing file's details: name, type, ICE, IF, RC, patente, T.S.C, capital, CNSS, address.

1. Check the sender is allowed to work on that client file.
2. Change only the fields the client asked for, to the values the client wrote. The AI cannot fill in or “correct” a value.
3. A new ICE or IF is checked against the official register; skipping that check needs the client's explicit confirmation.

> **Short and fast:** nothing waits on Experio here.

Built in

- **One firm, one assistant.** Several firms share one installation, but each has its own agent, mailbox, data and Experio access. A firm's agent cannot reach another firm's data.
- **Test mode per firm.** Drafts are created but nothing is posted. GECAFISC is currently live.
- **No lost or doubled work.** A run that fails is retried; after 3 failures the thread is set aside with the reason. A reply is never sent twice, and an interrupted run picks up where it stopped.
- **AI model:** MiniMax-M3, about 8 seconds per answer on average.

Current limits

- **5-minute cap.** Each piece of work the AI starts may run up to 5 minutes (raised from n8n's 2). Tarik stops itself in time and finishes anything longer by follow-up email.
- **Experio's reading speed.** Scanned invoices can stay unread in Experio for a long time (over an hour in today's test). Tarik handles the wait but cannot speed it up.
- **Email is the main channel.** The n8n chat works for testing but takes fewer file types and has no follow-up. WhatsApp could reuse the follow-up later.

State as of 16 October 2026 · n8n 2.43 · tested on the GECAFISC firm

← TARIK WORKFLOWS OVERVIEW

Tarik - Emails

Answers each firm's Tarik mailbox. Woken by the mail watcher whenever an email thread waits for an answer, it hands each thread to the firm's AI agent and sends the reply in the same thread. When work was left waiting, it sends a follow-up email later, without the agent.

STARTED BY

The mail watcher, through a protected webhook, naming the firm.

WORKS ON

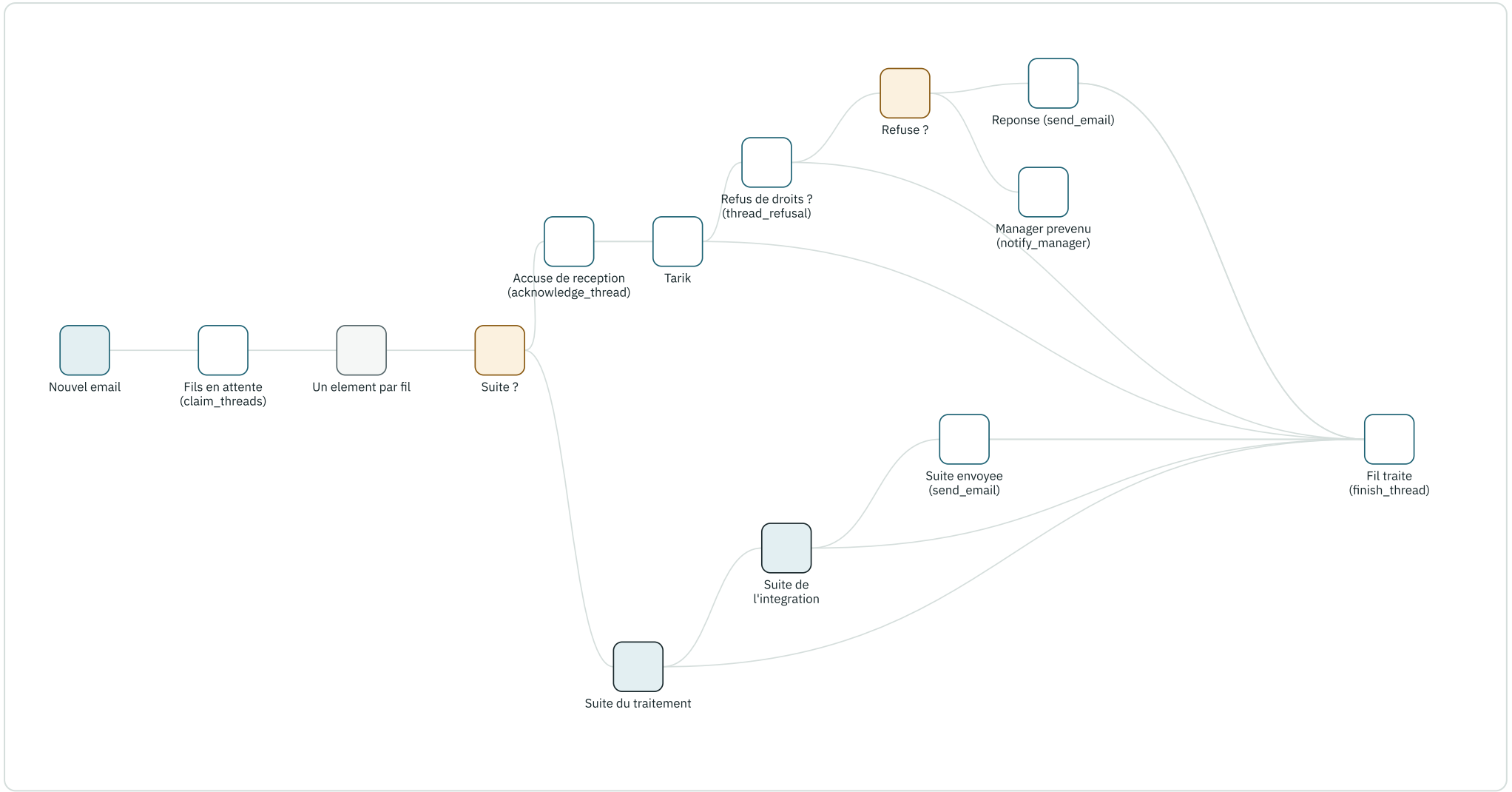
Every email thread of that firm waiting for an answer, one at a time.

ENDS WITH

A reply or a follow-up in the thread, and the thread's new state.

The workflow in n8n

Each box is a node, named exactly as in n8n's editor and placed where it sits there. Lines show where each node leads.



Start · Tarik tool (one Experio action) · Decision · Sorting (no Experio action) · Runs another workflow

Picking up the threads

`Nouvel email`	The webhook the mail watcher calls when a thread waits: a new email arrived, or a follow-up is ready. It only answers calls carrying the shared secret, and it names the firm.
`Fils en attente (claim_threads)`	Takes the firm's threads waiting for an answer and locks them for this run, so two runs never answer the same thread.
`Un element par fil`	Splits them up: each thread goes through the rest on its own.
`Suite ?`	A thread with new emails from the client goes to the agent. A follow-up (nothing new from the client, work left waiting) skips the agent.

Answering with the agent

`Accuse de reception (acknowledge_thread)`	A new conversation (its first email, not a reply in an earlier thread) gets an instant reply in the thread: “Votre message est bien reçu. Je le traite et vous réponds dans ce fil, en général d'ici quelques minutes.” Once per thread, never for a reply or a follow-up. If it cannot be sent, the agent answers all the same.
`Tarik`	Asks the firm's AI agent to answer the thread's new emails, in the thread's own conversation, with its attachments. The agent calls Tarik - Traitement, Tarik - Dossier or Tarik - Modification. Waits up to 11 minutes for the answer.
`Refus de droits ? (thread_refusal)`	Checks whether the work was refused because the sender is not allowed on that client file.
`Refuse ?`	Allowed: the answer is sent. Refused: the sender gets nothing and the firm's manager is told.
`Reponse (send_email)`	Sends the agent's answer as a reply in the thread, only to approved addresses.
`Manager prevenu (notify_manager)`	Emails the firm's manager that this sender wrote and was refused, at most once a day per sender.

Following up

Only for a thread whose work was left waiting: invoices Experio was still reading, or a new client file still onboarding. No agent, and no time limit.

`Suite du traitement`	Runs Tarik - Traitement in resume mode: the conversation's invoices left waiting are taken up again.

`Suite de l'integration`	Runs Tarik - Dossier in resume mode: the conversation's client files left waiting are finished, and their state is read.
`Suite envoyee (send_email)`	Sends both outcomes in one email in the same thread, starting “Suite du traitement de vos documents :”. A part with nothing to do is left out.

Closing the thread

`Fil traite (finish_thread)`	Marks the emails answered and sets the thread: done, waiting for the client's answer to a question, or followed up while work waits (30 minutes at most). When something failed (the reply could not be sent, the agent did not answer), the thread is set aside with the reason. Retried 3 times, so a reply is never sent twice.

State as of 19 October 2026 · node names as in n8n's editor

← TARIK WORKFLOWS OVERVIEW

Tarik - Traitement

Turns the documents a client sent into entries in Experio: bank statements imported, invoices drafted, checked, completed and posted, one fixed step after another. When a step needs the client, it asks one question and waits for the answer.

STARTED BY

The AI agent (Experio_traiter_fichiers to process files, Experio_repondre for an answer), or Tarik - Emails following up.

TIME BUDGET

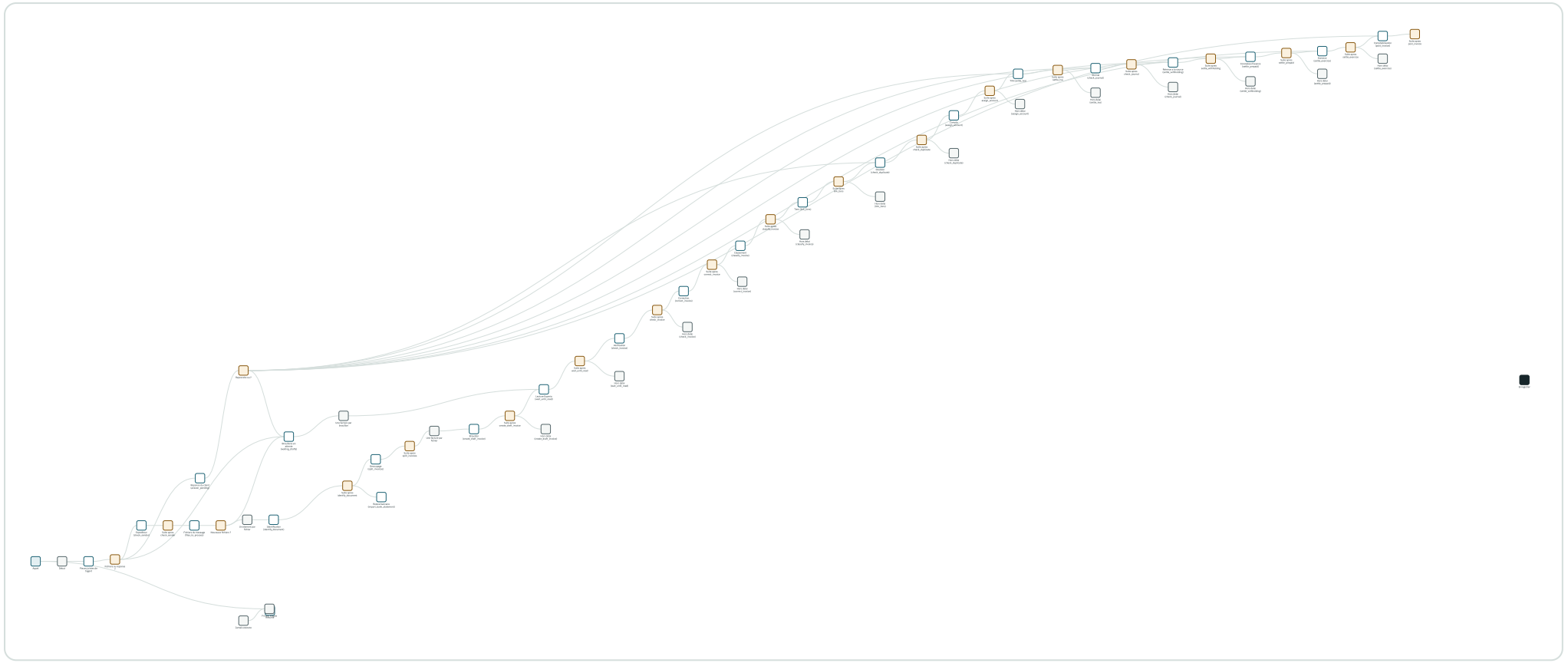
240 seconds, under the 5 minutes n8n allows. What is left waits for the next run. A follow-up has no budget.

ENDS WITH

A summary invoice by invoice, which the agent copies into its answer.

The workflow in n8n

Each box is a node, named exactly as in n8n's editor and placed where it sits there. Lines show where each node leads.



Start · Tarik tool (one Experio action) · Decision · Sorting (no Experio action) · Record

Every step also records its outcome in Enregistrer; those lines are left out.

Scaled to fit the page: the HTML version shows this map full size.

Starting a run

`Appel`	The call and what comes with it: the mode (files, answer or resume), the conversation, the client file, purchase or sale as the client said, the client's message, the attachments, the firm.
`Debut`	Notes when the run started, for its time budget.
`Pieces jointes de l'agent`	Saves files attached in n8n's chat. Files sent by email were saved when the email arrived.
`Fichiers ou reponse ?`	Files to process, the client's answer to a question, or a follow-up.

Files: from document to draft

`Expediteur (check_sender)`	The email's sender must be one of the firm's staff, or of Experio's own team, with this client file assigned. Otherwise the run stops there.
`Fichiers du message (files_to_process)`	The conversation's files not processed yet.
`Nouveaux fichiers ?`	None new (“relancez le traitement”): the invoices left waiting are taken up instead.
`Un element par fichier`	One item per file.
`Identification (identify_document)`	Invoice or bank statement?
`Releve bancaire (import_bank_statement)`	Imports a bank statement into the client file's bank journal. With several bank accounts, asks the client which one.
`Decoupage (split_invoices)`	Finds each invoice inside a PDF. Needs purchase or sale from the client; otherwise it asks, and nothing is created.
`Une facture par fichier`	One item per invoice found.
`Brouillon (create_draft_invoice)`	Creates the draft invoice in Experio, never twice for the same file.

Each invoice: the fixed sequence

Every invoice goes through these steps in this order. Each one either goes through, asks the client one question, or stops the invoice.

`1. Lecture Experio (wait_until_read)`	Waits for Experio's own reading (OCR) of the draft: 3 minutes at most for all the run's drafts together.
`2. Verification (check_invoice)`	Compares what Experio captured with what the scan shows, and lists every difference. Changes nothing.
`3. Correction (correct_invoice)`	Writes what the scan shows over what Experio captured differently: amounts, number, date, credit note, several VAT rates, currency, payment method.
`4. Classement (classify_invoice)`	Purchase, sale or in fact a bank statement. A scan that contradicts what the client said stops the invoice.
`5. Tiers (link_tiers)`	Sets the supplier or customer Experio detected, with the account it needs. When none was detected, asks the client, suggesting close names.
`6. Doublon (check_duplicate)`	Already recorded in this client file (same number, supplier or customer, date)? A duplicate stops here.
`7. Compte (assign_account)`	Sets the account: the one Experio remembers for this supplier, else the one matching what is billed (e.g. 6125). When none fits, asks the client.
`8. TVA (settle_tva)`	Recoverable or not, the firm's prorata, the tax on each line. Asks when the VAT might not be recoverable.
`9. Journal (check_journal)`	Puts the invoice in the journal Experio's rules give it, or names the missing setting.
`10. Retenue a la source (settle_withholding)`	Withholding tax on a purchase: the supplier's known rule, else a question.
`11. Constatee d'avance (settle_prepaid)`	An invoice covering several months: asks whether to spread it (charges or produits constatés d'avance).
`12. Exercice (settle_exercice)`	Books the invoice in an open fiscal year. When none covers its date, asks the client.

`13. Comptabilisation (post_invoice)`	Posts the invoice when Experio's own check finds nothing blocking; otherwise lists what blocks it.

Between the steps

Two kinds of nodes repeat after every step of the sequence:

`Suite apres <step>`	After each step, a decision. The step went through: on to the next step. It asked the client a question, stopped the invoice (a duplicate, a missing setting) or failed: the outcome is recorded and this invoice goes no further in this run.
`Hors delai (<step>)`	The run has used its 240 seconds: the invoice is recorded as waiting at this step instead of starting the next one. The client's next message, or the email follow-up, takes it up again from Experio's reading.

The client's answer

`Reponse du client (answer_pending)`	Applies the client's answer, in their own words, with what the agent read in it (oui or non, the supplier chosen among those proposed), to the question this conversation waits on: a supplier, an account, VAT, withholding, prepaid, fiscal year, or a statement's bank account. An unclear oui or non is asked again. Checks the sender first.
`Reprendre ou ?`	Resumes the invoice at the step after the answered one (Doublon, TVA, Journal, Retenue a la source, Constatee d'avance, Exercice or Comptabilisation). With nothing waiting, takes up the invoices left waiting.

Taking up waiting invoices

`Brouillons en attente (waiting_drafts)`	The conversation's drafts left waiting: Experio was still reading them, a step failed, or a run was cut short. Only those still drafts in Experio.
`Une facture par brouillon`	Each one goes back to Lecture Experio, and every later step runs again.

Recording and answering

`Enregistrer`	Records every step's outcome for every invoice. A question becomes the invoice's pending question, which the client's answer settles.
`Fin des etapes`	Waits for every step to end (its second input, from Jamais execute, never comes), then lets Résumé run.
`Jamais execute`	Never runs: it only keeps Fin des etapes waiting until the end.
`Résumé`	After Fin des etapes: the run's summary, invoice by invoice and step by step. It is the workflow's answer, which the agent copies.

State as of 19 October 2026 · node names as in n8n's editor

← TARIK WORKFLOWS OVERVIEW

Tarik - Dossier

Opens a new client file (dossier) in Experio, optionally copies another file's setup, imports the client's accounting history (FEC or general ledger), finishes the onboarding and reports where the file stands. Also loads a FEC into an existing file, or (update mode) changes an existing file's details: name, type, ICE, IF, RC, patente, T.S.C, capital, CNSS, address.

STARTED BY

The AI agent (Experio_parametrer_dossier, Experio_modifier_dossier), or Tarik - Emails following up.

TIME BUDGET

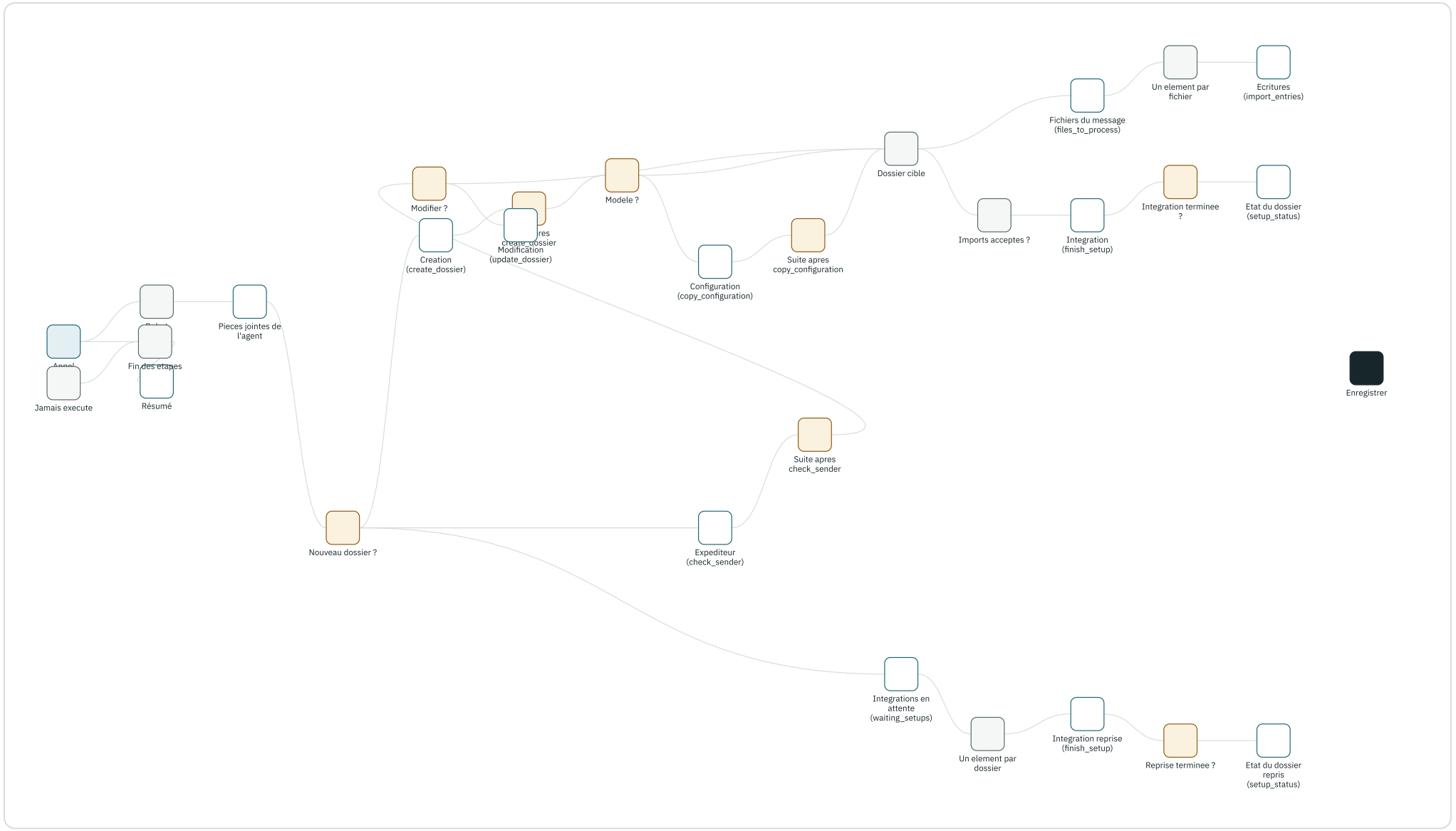
260 seconds. When Experio's wait would not fit, the file is left waiting and the email follow-up finishes it.

ENDS WITH

A summary: creation, import, onboarding and the file's state.

The workflow in n8n

Each box is a node, named exactly as in n8n's editor and placed where it sits there. Lines show where each node leads.



Start · Tarik tool (one Experio action) · Decision · Sorting (no Experio action) · Record

Every step also records its outcome in Enregistrer; those lines are left out.

Starting a run

`Appel`	The call: the conversation, the client file (0 for a new one), its name, ICE, IF, company or person, the file whose setup to copy, FEC or general ledger, the attachments, the firm.
`Debut`	Notes when the run started, for its time budget.
`Pieces jointes de l'agent`	Saves files attached in n8n's chat.
`Nouveau dossier ?`	A new client file, an existing one, a change to one, or a follow-up.

A new client file

`Creation (create_dossier)`	Creates the file as Experio's Folders screen does. Experio checks the name is free and the ICE and IF against the official register. A very similar name, or skipping the register check, is confirmed with the client first.
`Suite apres create_dossier`	Created: on to the next step. A question or a refusal: recorded, and the run stops there.
`Modele ?`	Copy another file's setup only if the client asked for it.
`Configuration (copy_configuration)`	Copies that file's accounts, taxes, journals, bank operations and settings, before any import.
`Suite apres copy_configuration`	Copied: on to the import. Otherwise recorded, and the run stops.

An existing client file

`Expediteur (check_sender)`	The email's sender must be one of the firm's staff, or of Experio's own team, with this client file assigned.

`Suite apres check_sender`	Allowed: on to the change or the import. Otherwise the run stops.

Importing the history

`Dossier cible`	The client file the rest works on: the one given, or the one just created.
`Fichiers du message (files_to_process)`	The conversation's FEC and general ledger files not imported yet.
`Un element par fichier`	One item per file.
`Ecritures (import_entries)`	Imports the entries through Experio's Importation screen, after Experio's own test of the file. Experio creates the missing accounts, journals, suppliers, customers and fiscal years. A refused file is reported; the client sends a corrected one.

Changing a client file

Only in update mode (Experio_modifier_dossier).

`Modifier ?`	A change: on to it. Otherwise the existing file is set up (imports below).
`Modification (update_dossier)`	Changes only the fields given, as the client file's Experio form does, and only to values the client wrote in this conversation. An ICE, IF or name another file already has is refused. A new ICE or IF is checked against the official register; skipping that check needs the client's confirmation first.

Finishing the onboarding

`Imports acceptes ?`	Only when no import was refused.
`Integration (finish_setup)`	Completes Experio's onboarding steps so Experio processes the imported entries, opening the current fiscal year if none is open. Experio only takes lines once they are a minute old: when that wait does not fit in the run's time, the file is left waiting for the follow-up.

`Integration terminee ?`	Finished: read the file's state. Left waiting: its state comes with the follow-up.
`Etat du dossier (setup_status)`	Where the file stands: imported entries by state, fiscal years, and any setting Experio needs before an invoice can be posted.

Following up

Only when Tarik - Emails follows up a thread.

`Integrations en attente (waiting_setups)`	The conversation's client files whose entries were imported and whose onboarding still waits.
`Un element par dossier`	One item per file.
`Integration reprise (finish_setup)`	Finishes the onboarding, with no time limit.
`Reprise terminee ?`	Finished: read the file's state.
`Etat du dossier repris (setup_status)`	Where the file stands, for the follow-up email.

Recording and answering

`Enregistrer`	Records every step's outcome.
`Fin des etapes`	Waits for every step to end (its second input, from Jamais execute, never comes), then lets Résumé run.
`Jamais execute`	Never runs: it only keeps Fin des etapes waiting until the end.
`Résumé`	After Fin des etapes: the run's summary, which the agent copies.

State as of 19 October 2026 · node names as in n8n's editor

Tarik repository guide

For a developer joining the project: what Tarik is, every file and folder of the repository and what it is for, and the day-to-day loop. The README is the reference; this is the tour.

What Tarik is

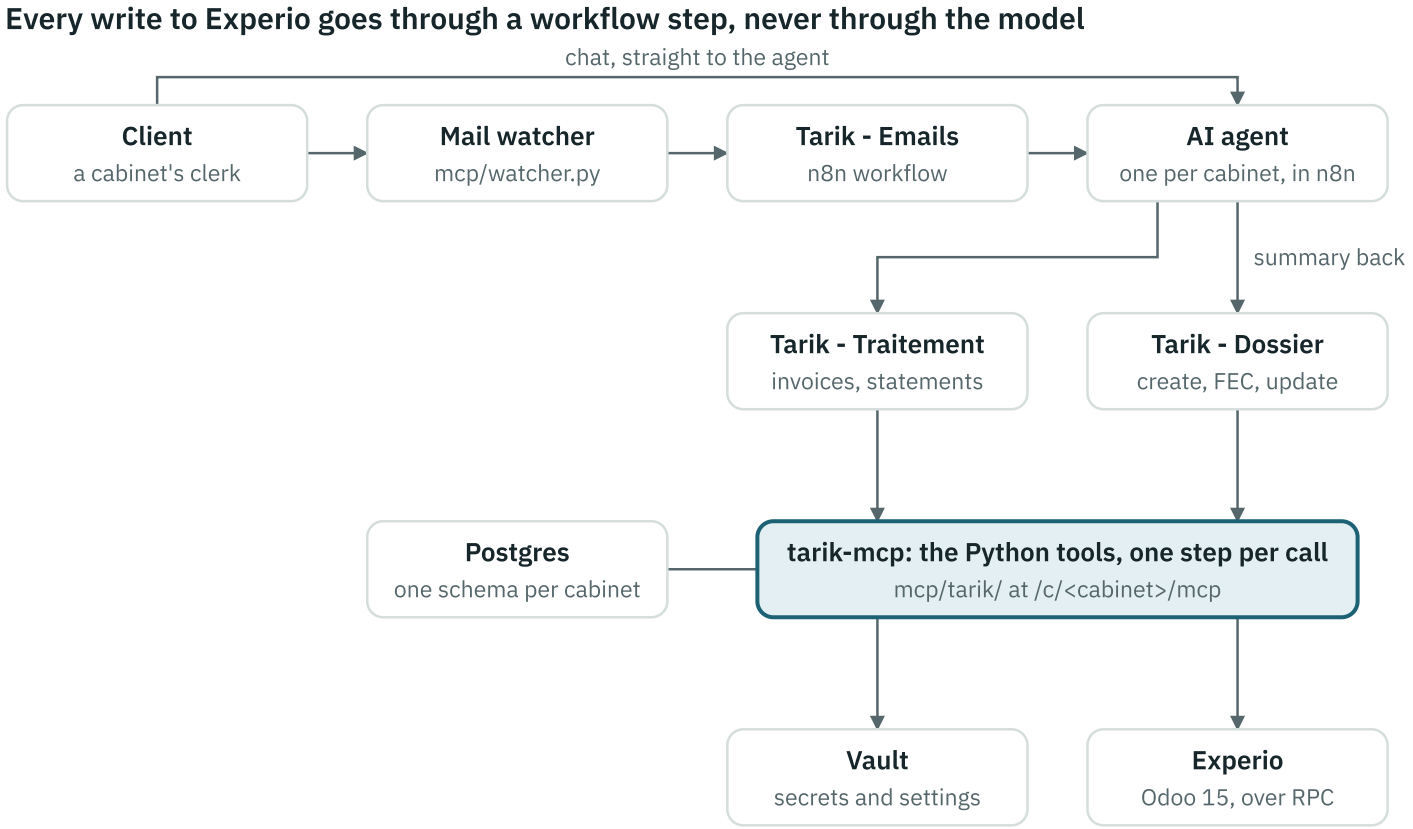
Tarik is an accounting assistant for Moroccan cabinets (accounting firms) using Experio, an Odoo 15 app. A clerk writes to Tarik by chat or email, attaches invoices, bank statements or a FEC, and Tarik puts them into Experio through Experio's own screens and wizards. Nothing is invented: a model (MiniMax-M3) only reads scans and the client's words; every write is a fixed Python step.

The pieces, each a Docker service in `docker-compose.yml`:

- **n8n** (patched, `n8n/`): hosts one native agent per cabinet ("Tarik - GECAFISC" and 7 more) and three workflows. The agent finds the dossier and hands the work to a workflow; the workflow calls the Python tools step by step.
- **tarik-mcp** (`mcp/`): the Python tools, served over MCP at `http://tarik-mcp:8000/c/<cabinet>/mcp`. One address per cabinet, so a call can only ever work in the cabinet it named.
- **tarik-mail-watch** (`mcp/watcher.py`, same image): watches each cabinet's mailbox over IMAP IDLE and wakes the email workflow.
- **postgres**: Tarik's own data (one schema per cabinet, from `mcp/tarik/core/cabinet.sql`) and n8n's data.
- **vault**: every secret and per-cabinet setting (`secret/tarik/shared`, `secret/tarik/cabinets/<cabinet>`). The cabinets are those Vault paths.

A cabinet is named by its exact Experio database (`experio_cabinet_8`). That name is its MCP address, its Vault path and its Postgres schema. The agent never sees Odoo, a URL or a setting: it is "Experio" everywhere.

A request flows like this: an email arrives, the watcher stores it and wakes "Tarik - Emails"; that workflow asks the cabinet's agent; the agent calls "Tarik - Traitement" (invoices and statements) or "Tarik - Dossier" (a new dossier, a FEC, or a change to a dossier); each node of those workflows is one MCP tool in `mcp/tarik/`; the run's summary comes back to the agent, then to the client. Chat follows the same path without the watcher and the email workflow.



The agent only chooses which workflow to call and fills a few fields; each workflow node is one Python tool, and only those tools write to Experio. Postgres also holds n8n's own data.

Repository layout

Everything at the root, and what each entry is for.

ENTRY	WHAT IT IS
`README.md`	The reference: how it is built, the rules the tools keep, cabinets and Vault, setup, deploy, tests. Read it first.
`docker-compose.yml`	The five services above. `tarik-mcp` gets only the variables Tarik reads, never the whole `.env`.
`.env` / `.env.example`	Host settings: Postgres names and password, `ODOO_URL`, n8n's encryption key, `N8N_MCP_API_URL`. `.env` is ignored by git; copy the example.
`pyproject.toml`	Ruff's formatter and linter settings: Python 3.14, 120 columns. A style test enforces them on every Python file.
`.gitignore`	`.env`, `vault/.local/` (unseal key, root token, AppRole ids), `.temp/`, caches.
`mcp/`	The Python side: the MCP server, the mail watcher, the `tarik` package and its tests. Built into the `tarik-mcp` image.
`workflows/`	The n8n workflows as JSON, the Python builders that generate them, and the agent template.
`instructions/`	Everything the model is told: the agents' skills and the prompts the tools send.
`n8n/`	The n8n image: a Dockerfile and `patch.js`, which edits a few lines of n8n at build time.
`vault/`	Vault's config and the two scripts that start, unseal and set it up locally.
`docs/`	Three Markdown docs, this guide, and a generated HTML/PDF overview of the workflows for the project team.
`.claude/`, `.temp/`	Local tooling and scratch; not part of the project.

mcp/: the Python tools

Every tool is a Python function decorated with `@server.tool()` (or `@step_tool`, which also copies the invoice, file and dossier ids into the result so n8n can route on them). Importing a module registers its tools. A tool returns a dict with `status` (`ok`, `wait`, `question`, `stop`) and a French `message` the client reads; a `ToolError` is the only text that reaches the agent on failure.

Entry points and packaging

FILE	ROLE
`mcp/server.py`	The ASGI app: routes `/c/<cabinet>/mcp` to the MCP server inside `cabinets.using(name)`, with a semaphore per cabinet and one for the whole server. At start it refuses to run unless Vault is set, every cabinet logs into Experio with the expected user id, and each schema exists.
`mcp/watcher.py`	`tarik-mail-watch`: one IMAP IDLE thread and one 15-minute tick thread per cabinet mailbox. Stores new emails, tells the manager of refused senders, and posts to the "Tarik - Emails" webhook with a shared secret header.
`mcp/Dockerfile`, `.dockerignore`, `pytest.ini`	Python 3.14 slim with pinned dependencies (mcp, psycopg, pillow, pypdfium2, httpx, openpyxl, pytest, ruff). `pytest.ini` declares the test categories as markers and excludes `live` and `llm` by default.

** `tarik/core/` : what every tool builds on**

MODULE	ROLE
`app.py`	The `MCPServer` instance, `log()`, the result makers `ok` / `wait` / `question` / `stop` / `routine`, and the `with_ids` and `step_tool` decorators.
`config.py`	Constants shared by all cabinets: `ODOO_URL`, Postgres, timeouts, Experio's state codes, the dossier domain, name-matching thresholds, PDF rendering settings.
`cabinets.py`	The Vault client (AppRole login, settings re-read every 5 minutes, last values kept on an outage), the list of cabinets, `current()` and `using()` (a context variable holding the call's cabinet), `setting(key)` which refuses a key in the wrong place, `cabinet_db()` (a Postgres connection whose search_path is that cabinet's schema only), and `create_schemas()`.
`cabinet.sql`	One cabinet's tables, idempotent: `tarik_files`, `tarik_checks`, `tarik_classifications`, `tarik_results`, `tarik_pending`, `tarik_setups`, the mail tables, `tarik_client_messages`. Columns added later use `alter table ... add column if not exists`.
`experio.py`	Experio calls and guards: XML-RPC (`odoo_call`, one proxy per thread) and JSON-RPC (`odoo_json`, for answers holding None), login checked against `ODOO_UID`, `dry_run()` and `TEST_MODE`, the `refused()` context manager that turns any failure into a French ToolError, `require_dossier`, `require_invoice`, `require_draft_invoice`, `fill_invoice` (types fields into the form one by one so Experio's onchanges run), `refresh`, `experio_notes`, and the `experio_connection` tool.
`model.py`	The vision model over the Anthropic message format MiniMax imitates: `ask_model`, `json_answer` (the one JSON object in a reply), `instruction(name)` reading `instructions/tools/<name>.md`, and the PDF page rendering (`pdf_pages`, `scan_pages`, `to_pdf`).
`words.py`	Reading what people write: `normalized`, `said`, `similarity`, `ranked`, `strip_tags`, `ICE_WRITTEN`, and numbers in French or American style (`readings`, `as_number`, `written_numbers`).
`conversation.py`	The client's own words per conversation (`client_messages`, `own_words`), `yes_no` and `chosen_among`: a choice is read from the agent's decision, never guessed.

** `tarik/dossiers/` : dossiers and people**

MODULE	TOOLS
`search.py`	`list_dossiers`, `find_dossier` (fuzzy name match, 5 closest with a score).
`people.py`	`list_collaborators`, `check_sender`: an email's sender must be a collaborator or Experio staff with the dossier assigned.
`identity.py`	`create_dossier`, `update_dossier`: identity fields only to values the client wrote themselves, duplicates by ICE, IF or name refused, confirmation questions before skipping the registry check.
`setup.py`	`copy_configuration`, `import_entries` (FEC or grand livre through Experio's import screen), `finish_setup` (walks the onboarding stepper, waits for lines to be a minute old), `setup_status`, `waiting_setups`.

** `tarik/invoices/`: the fixed sequence, one module per step**

MODULE	TOOLS
`intake.py`	`identify_document`, `split_invoices` (one PDF into several), `create_draft_invoice`, `import_bank_statement`. Rows are locked `for update` so a retry never creates a second draft.
`classify.py`	`classify_invoice`: achat, vente or bank statement; also `side_of` and `client_side` used by other steps.
`check.py`	`check_invoice`: the model reads the scan and compares with what Experio captured; dates resolved in code; false alarms dropped. Stores the report in `tarik_checks`.
`correct.py`	`correct_invoice`: writes the differences back, plus credit note, multi-TVA split, currency, payment method.
`tiers.py`	`link_tiers`, `find_tiers`, `choose_tiers`: the supplier or customer, Experio's detection first, else a question listing close names.
`duplicates.py`	`check_duplicate`: Experio's own check against posted invoices and the dossier's other drafts.
`accounts.py`	`assign_account`, `set_account`: the memorized account, else the model picks among the dossier's real chart, else a question.
`tva.py`	`settle_tva`: recoverable or not (a question on fuel, travel and car accounts), the cabinet's prorata, Experio's tax lines.
`journal.py`	`check_journal`: the journal Experio's rule gives, or which dossier setting is missing.
`withholding.py`	`settle_withholding`: retenue à la source, the tiers' known rule or a question read by the model.
`prepaid.py`	`settle_prepaid`: charges constatées d'avance when the billed period spans months.
`exercice.py`	`settle_exercice`: the fiscal year, relinking, late invoices, or opening a new year.
`posting.py`	`post_invoice`: only when Experio's own validation is clear; state read back.

MODULE	TOOLS
`listing.py`	`list_invoices`: a dossier's invoices by status, for the agent.
`pipeline.py`	The run's bookkeeping, used by n8n rather than the agent: `wait_until_read`, `record_step`, `run_summary`, `answer_pending`, `save_files`, `files_to_process`, `waiting_drafts`. `STEPS` is the sequence in order.

** `tarik/emails/`: the mailbox**

MODULE	ROLE
`inbox.py`	Reads IMAP without changing it, threads emails by `In-Reply-To` and `References`, refuses non-collaborators and auto-replies, and the thread tools the email workflow calls: `claim_threads`, `acknowledge_thread`, `thread_refusal`, `notify_manager`, `finish_thread`.
`sending.py`	`send_email`: whitelist only (`MAIL_ALLOWED_RECIPIENTS`), pinned certificate, Markdown rendered to text and HTML, a copy saved in Sent.
`agent.py`	`ask_agent`: asks the cabinet's n8n agent through n8n's MCP `call_agent`, in the thread's own conversation.

mcp/tests/: the tests

Tests run in a throwaway `tarik-mcp` container on your working copy, against the test database `tarik_test` (never the cabinets' own) with settings from the environment instead of Vault. Each test has one category marker: `security`, `functionality`, `communication`, `technical`, `style`, plus `live` (the real n8n) and `llm` (the real model, slow, billed), which only run when asked.

FILE OR FOLDER	ROLE
`run.py`	The runner. `python3 mcp/tests/run.py` runs the tests covering the files changed since the last commit (it follows imports); `--all` runs everything; `-m llm` a category. Any other argument goes to pytest.
`conftest.py`, `testdb.py`, `fakes.py`	The test database and a throwaway cabinet schema per run (`db_cabinet` fixture, emptied around each test); `FakeDb` and `flag` stand-ins.
`runs.py`	A realistic ten-invoice run, recorded as n8n records it, for the summary tests.
`core/`	Experio guards and the proxy cache (`test_experio.py`), Vault against a fake server (`test_vault.py`), the test database itself.
`dossiers/`	Search, people, dossier creation and update rules, onboarding, the setup wait.
`invoices/`	One file per step (`test_check.py`, `test_tiers.py`, `test_duplicates.py`, `test_tva.py` ...), the answer routing (`test_answers.py`), the summary, the files. `replies/` holds real model replies that once broke the JSON reader.
`emails/`	Threads, inbox reading, sending and whitelist, the agent call, the watcher.
`isolation/`	The security suite: one cabinet never sees another's schema, settings or HTTP address, and no tool ever shows a setting, whatever Experio, the mailbox or the model do (`test_no_setting_shown.py` calls every tool).
`style/`	Ruff lint and format on every Python file, each workflow JSON equal to what its builder produces, every tool a workflow or skill names exists, every caller fills every workflow input, each prompt file read by the tool it names.
`n8n/`	Live: the agent's memory and conversation, and `test_at_the_end.py` which runs "Tarik - Test Merge" to check n8n's execution order after an upgrade.
`llm/`	Live, measured on many replies: `test_decisions.py` (what the agent reads as oui/non, which tiers, exercice or bank account the client chose) and `test_relay.py` (every fact of a summary reaches the client).

workflows/: n8n

FILE	ROLE
`steps.py`	What the builders share: the `Workflow` class (nodes, connections, an MCP step with its cabinet address, a router, a code node, `record_and_summary`, `at_the_end`), and the input lists `TRAITEMENT_INPUTS` and `DOSSIER_INPUTS` every caller must fill.
`build_tarik_traitement.py` / `tarik-traitement.json`	"Tarik - Traitement" (`TarikTraitement1`): the invoice sequence, one node per step, each only when the previous said ok. Modes `files` (new files), `answer` (the client's answer, resumed at the right step), `resume` (drafts Experio was still reading, no agent). A 240-second budget: invoices still going wait for the next run.
`build_tarik_dossier.py` / `tarik-dossier.json`	"Tarik - Dossier" (`TarikDossier0001`): create, copy a model's configuration, import each FEC, finish the onboarding, report the state. Mode `update` changes an existing dossier's identity fields instead. Mode `resume` finishes dossiers left waiting.
`build_tarik_emails.py` / `tarik-emails.json`	"Tarik - Emails" (`TarikEmails00001`): a webhook woken by the watcher; claims the waiting threads, acknowledges a new one, asks the cabinet's agent, sends the reply; a follow-up runs Traitement then Dossier in mode resume and emails both summaries.
`build_tarik_merge_test.py` / `tarik-merge-test.json`	"Tarik - Test Merge": only for `tests/n8n/test_at_the_end.py`, which checks n8n still runs the summary node last after an upgrade.
`tarik-agent.json`	The agent template: instructions, the five read-only MCP tools, and four workflow tools (`Experio_traiter_fichiers`, `Experio_repondre`, `Experio_parametrer_dossier`, `Experio_modifier_dossier`) with their fixed and model-filled inputs. `__CABINET__` and `__CABINET_NAME__` are replaced per cabinet.
`build_tarik_agents.py`	Creates or updates every cabinet's agent from the template and the skills, through n8n's MCP server. The cabinets are listed in its `CABINETS` dict. It refuses to build when a skill names a tool the agents do not have. Prints each `AGENT_ID` for Vault.
`layout.json` / `save_layout.py`	Node positions as arranged in n8n's editor, read from n8n's database by the script and applied by the builders, so a rebuild keeps the layout.

instructions/, n8n/, vault/, docs/

** `instructions/` : what the model is told, written once**

FILE	ROLE
`skills/nouveau-dossier.md` , `traitement-documents.md` , `modification-dossier.md` , `reponse-question.md`	The agents' four skills (front matter `name` , `description` , then the instructions, in French). `build_tarik_agents.py` gives every agent the same copy and removes a skill from all agents when its file goes. They name tools exactly as the model sees them (`Experio_find_dossier`).
`tools/check-invoice.md` , `check-invoice-answer.md`	What `check_invoice` tells the model: how to read a Moroccan invoice, and the JSON shape to answer with.
`tools/classify-document.md`	Achat, vente or releve bancaire, for `identify_document` and `classify_invoice` .
`tools/split-pdf.md`	Which pages begin a new invoice, for `split_invoices` .
`tools/choose-account.md`	The CGNC accounts and the rules, for `assign_account` .

Each prompt file starts with front matter `used_by` ; a style test checks it is read by that tool. The container mounts the folder, so a prompt change applies on restart with no rebuild. Short answer shapes (a few lines) live as constants next to the code that reads the answer.

** `n8n/` : the patched n8n image**

FILE	ROLE
`Dockerfile`	n8n 2.43.5 plus the patch.
`patch.js`	Edits n8n's own source at build time: an agent's workflow tool may run `TARIK_AGENT_TOOL_TIMEOUT_MS` (5 minutes, n8n's fixed value is 2); a tool input fixed to `{{threadId}}`, `{{attachments}}` or `{{clientMessage}}` is filled by n8n with the conversation's id, its files and the client's latest message word for word, never by the model; `call_agent` accepts `newSession`. The build fails if an n8n upgrade moves a patched line.

** `vault/` : secrets, local setup**

FILE	ROLE
`config.hcl`	Vault's file storage config.
`start.sh`	The container's entrypoint: starts Vault and unseals it with the key kept in `vault/.local/unseal/`.
`setup.sh`	Run once: creates the KV mount, the policy that only reads `secret/tarik/*`, and the AppRole whose `role_id` and `secret_id` are written to `vault/.local/approle/` and mounted read-only into `tarik-mcp`.

`vault/.local/` is git-ignored: unseal key, root token and AppRole ids live on the machine only. On the public server a real Vault takes this container's place.

** `docs/` : documentation**

FILE	ROLE
`architecture.md`	The system diagram, the trust rules (what the model may decide, what only the client's words may set), the data each service keeps.
`tools-and-workflows.md`	Every tool, skill and workflow in one table each, and the scripts.
`using-tarik.md`	For the cabinet: what Tarik can do and how to write to it.
`repository-guide.md`	This guide; `workflows-overview/build.py` renders it to `guide.html` and `guide.pdf`.
`workflows-overview/build.py`, `overview.html`	A generated site for the project team: `index.html` from the hand-written `overview.html`, one page per workflow with a map drawn from the JSON and every node explained, this guide, then PDFs with Chrome and `pdfunite`. The generated `.html` and `.pdf` files are committed. Run it after a workflow changes; it fails when a node has no explanation.

Day to day

Rules the code keeps, and tests enforce

- Every dossier tool needs a real dossier id from `find_dossier`; every invoice tool a real draft of one of the cabinet's dossiers.
- The model never decides a value: an ICE, an amount, a name is taken only when the client wrote it; a choice only among what the question proposed.
- `DRY_RUN=1` per cabinet (the default): drafts and statements are created, nothing else is written; replies start with `MODE TEST`.
- A setting is either the cabinet's own or shared, never both, and never shown to the agent.
- One module per invoice step; a step tells the client what it did in one French sentence; routine outcomes are left out of the summary.

Changing Python

1. Edit a module under `mcp/tarik/`.
2. `python3 mcp/tests/run.py` runs the tests that cover your change; `--all` for everything. Ruff lint and format are part of the style tests.
3. `docker compose up -d --build --force-recreate tarik-mcp tarik-mail-watch`.

Changing a workflow or the agents

1. Edit `workflows/build_tarik_*.py` or `workflows/steps.py`, run the builder.
2. `docker compose cp` the JSON into n8n, `n8n import:workflow`, `n8n publish:workflow --id=...`, `docker compose restart n8n` (README has the exact commands).
3. A change to `tarik-agent.json` or a skill: `python3 workflows/build_tarik_agents.py`. It updates all cabinets and republishes the agents that were published.
4. `python3 docs/workflows-overview/build.py` to regenerate the team overview.

Changing a prompt: edit `instructions/tools/*.md`, restart `tarik-mcp`.

Adding a cabinet: give Tarik a user in that cabinet's Experio, add its database name to `CABINETS` in `build_tarik_agents.py`, run it, put the agent id and the cabinet's settings in Vault under `secret/tarik/cabinets/<cabinet>`, restart `tarik-mcp` (its schema is made at start).

Where to look when something goes wrong: `docker compose logs tarik-mcp` (every Experio refusal is logged with the tool and the invoice), the run's rows in `tarik_results`, the thread's state in `tarik_mail_threads`, and n8n's execution of the workflow.
